

```
In [ ]: !apt-get -qq install imagemagick
!pip -q install scikit-learn --upgrade

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm
import seaborn as sns
import itertools
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.lines import Line2D
from IPython.display import Image
from scipy.spatial.distance import cdist

# Pyswarms solves the PSO https://pyswarms.readthedocs.io/en/latest/
try:
    import pyswarms as ps
except ModuleNotFoundError:
    !pip -q install pyswarms
    import pyswarms as ps
from pyswarms.utils.plotters import plot_cost_history, plot_contour, plot_surface
from pyswarms.utils.plotters.formatters import Animator, Designer, Mesher

from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error
from sklearn.model_selection import train_test_split

%matplotlib inline
```

```
E: Failed to fetch http://archive.ubuntu.com/ubuntu/pool/main/d/djvulibre/libdjvulibre-text_3.5.27.1-8ubuntu0.2_all.deb 404 Not Found [IP: 91.189.88.152 80]
E: Failed to fetch http://archive.ubuntu.com/ubuntu/pool/main/d/djvulibre/libdjvulibre21_3.5.27.1-8ubuntu0.2_amd64.deb 404 Not Found [IP: 91.189.88.152 80]
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-missing?
[K 22.3MB 1.4MB/s
[K 112kB 4.2MB/s
[?25h
```

Data from PSO paper

```
In [ ]: data = pd.read_csv("/content/drive/MyDrive/psa_data.csv", decimal=',', index_col=0)
columns_x = data.columns[data.columns != 'RK']
columns_y = data.columns[data.columns == 'RK']
data = data.apply(pd.to_numeric)
data['TC-min'] = - data['TC-min']
data['TC peak'] = - data['TC peak']
data_train, data_test = train_test_split(data, train_size=30, random_state=11)
data_train
```

Out [35]:

	PSD-10	Mois	DF * 10 ⁻⁴	TC-min	TC-max	TC peak	4SA	PSD-90	PSD-50	Sinter temp.	K	D-50	RK
42	0.51	0.01	31	-74.8	50.1	-5.6	3.0	1.87	0.97	1293	11.170	0.929	18.901
12	0.41	0.02	30	-74.0	43.5	-6.5	3.2	2.15	0.95	1283	10.797	0.843	17.292
7	0.43	0.06	29	-74.3	49.3	-7.4	3.0	1.97	0.92	1286	10.268	0.854	17.153
4	0.43	0.04	37	-73.8	44.5	-7.2	3.0	2.06	0.95	1292	10.345	0.910	16.188
20	0.46	0.05	29	-74.9	46.1	-6.5	3.0	1.93	0.94	1284	11.003	0.830	16.935
41	0.51	0.08	36	-74.8	47.3	-5.3	2.8	1.86	0.98	1266	10.858	1.008	17.747
21	0.43	0.08	39	-75.0	43.8	-5.7	3.1	2.01	0.94	1270	11.117	0.908	18.218
37	0.48	0.04	25	-76.4	59.8	-7.0	3.1	1.97	0.97	1293	11.804	0.873	19.840
22	0.46	0.01	35	-74.0	43.9	-6.8	3.1	1.93	0.94	1292	10.479	0.948	18.181
6	0.45	0.06	30	-74.5	47.5	-7.2	3.1	1.97	0.94	1282	10.806	0.905	16.642
11	0.42	0.01	30	-73.6	46.5	-6.9	3.1	2.12	0.96	1284	10.539	0.918	17.287
2	0.44	0.02	40	-73.6	41.8	-7.2	2.9	2.10	0.98	1307	11.142	0.995	17.145
31	0.46	0.06	32	-75.0	50.6	-5.8	3.2	1.91	0.93	1273	11.295	0.868	18.624
3	0.46	0.02	41	-74.5	42.1	-6.3	2.8	2.05	0.98	1271	10.834	0.940	16.066
5	0.45	0.06	28	-73.5	49.3	-9.0	3.0	1.97	0.94	1287	10.286	0.991	16.479
30	0.45	0.05	25	-74.6	54.0	-7.0	3.2	2.04	0.96	1270	10.994	0.914	17.052
39	0.47	0.03	30	-75.4	55.3	-6.3	2.9	1.94	0.96	1273	12.242	0.874	19.044
40	0.52	0.01	30	-75.1	51.2	-6.1	2.8	1.88	0.99	1288	11.312	0.918	17.427
33	0.50	0.02	27	-75.3	58.3	-7.0	3.2	1.82	0.95	1280	11.590	0.955	17.635
29	0.50	0.05	22	-74.8	54.0	-6.7	3.0	1.97	0.99	1274	11.008	0.846	17.308
25	0.43	0.02	25	-74.8	53.4	-6.7	3.1	2.00	0.93	1270	11.101	0.850	16.603
19	0.46	0.02	26	-74.0	47.1	-7.7	3.0	1.87	0.94	1300	10.508	1.020	17.537

	PSD-10	Mois	DF * 10 ⁻⁴	TC-min	TC-max	TC peak	4SA	PSD-90	PSD-50	Sinter temp.	K	D-50	RK
8	0.47	0.03	29	-74.4	49.3	-7.5	3.2	2.06	0.98	1274	10.904	0.876	15.201
34	0.49	0.01	25	-75.6	54.1	-6.4	2.9	1.83	0.95	1281	11.384	0.878	18.623
13	0.44	0.04	31	-74.2	45.7	-6.8	2.9	2.01	0.95	1272	10.846	0.824	18.302
14	0.45	0.02	30	-73.9	41.9	-6.5	2.9	2.04	0.98	1281	10.533	0.862	16.662
18	0.45	0.01	31	-74.2	45.3	-6.9	3.2	1.89	0.92	1296	10.651	0.883	17.781
28	0.44	0.01	28	-74.5	52.5	-6.8	3.0	1.98	0.94	1269	10.682	0.950	17.047
17	0.50	0.07	36	-74.0	41.1	-5.9	3.3	1.88	0.96	1281	10.232	1.000	18.030
26	0.47	0.01	40	-74.1	44.3	-5.2	3.0	1.92	0.95	1305	10.493	0.880	18.230

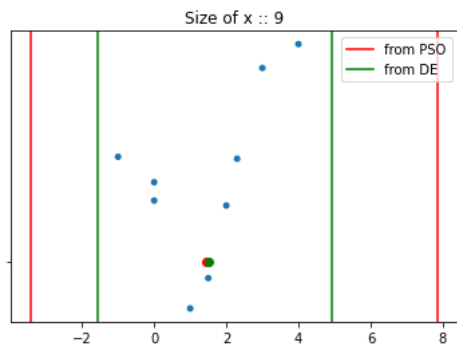
Tests

In []:

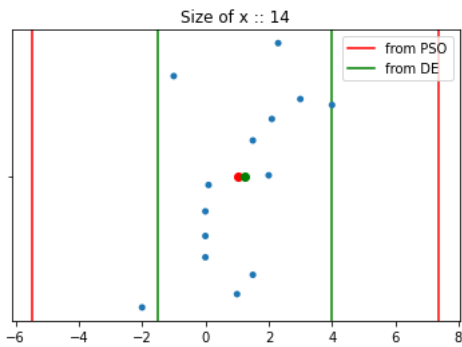
```
x = np.array([1,2,-1,0,0,1.5,4,3,2.3])
CL = x.mean()
N_L = np.count_nonzero(x < CL)
N_U = np.count_nonzero(x > CL)
sk_L = N_L / (N_L + N_U + 1)
sk_U = N_U / (N_L + N_U + 1)
LB = CL - 1/sk_U * (CL - x.min())
UB = CL + 1/sk_L * (x.max() - CL)
CL2 = np.median(x)
S_x = np.sum((x - x.mean())**2) / (x.shape[0] - 1)
#print(f"S :: {S_x} | var :: {x.var()}")
print(f"{np.sqrt(-2*S_x/N_L * np.log(10**-20))} || {(CL - x.min())}")
LB2 = CL2 - sk_L * np.sqrt(-2*S_x/N_L * np.log(10**-20))
UB2 = CL2 + sk_U * np.sqrt(-2*S_x/N_U * np.log(10**-20))
sns.stripplot(x=x)
plt.scatter(CL, 0, marker='o', c='r')
plt.axvline(LB, c='r', label='from PSO')
plt.axvline(UB, c='r')
plt.scatter(CL2, 0, marker='o', c='g')
plt.axvline(LB2, c='g', label='from DE')
plt.axvline(UB2, c='g')
plt.title(f"Size of x :: {x.shape[0]}")
plt.legend()
plt.show()
#####
x = np.array([1,2,-1,0,0,1.5,4,3,2.3, 0.1, 2.1, -2, 1.5,0])
CL = x.mean()
N_L = np.count_nonzero(x < CL)
N_U = np.count_nonzero(x > CL)
sk_L = N_L / (N_L + N_U + 1)
sk_U = N_U / (N_L + N_U + 1)
LB = CL - 1/sk_U * (CL - x.min())
UB = CL + 1/sk_L * (x.max() - CL)
CL2 = np.median(x)
S_x = np.sum((x - x.mean())**2) / (x.shape[0] - 1)
#print(f"S :: {S_x} | var :: {x.var()}")
print(f"{np.sqrt(-2*S_x/N_L * np.log(10**-20))} || {(CL - x.min())}")
LB2 = CL2 - sk_L * np.sqrt(-2*S_x/N_L * np.log(10**-20))
UB2 = CL2 + sk_U * np.sqrt(-2*S_x/N_U * np.log(10**-20))
sns.stripplot(x=x)
plt.scatter(CL, 0, marker='o', c='r')
plt.axvline(LB, c='r', label='from PSO')
plt.axvline(UB, c='r')
plt.scatter(CL2, 0, marker='o', c='g')
plt.axvline(LB2, c='g', label='from DE')
plt.axvline(UB2, c='g')
plt.title(f"Size of x :: {x.shape[0]}")
plt.legend()
plt.show()
#####
x = np.array([1,2,-1,0,0,1.5,4,3,2.3, 0.1, 2.1, -2, 1.5,0,1,2.4,0.7,-1.2,-1.3,-0.7,0.3,-2.1,-0.9])
CL = x.mean()
N_L = np.count_nonzero(x < CL)
N_U = np.count_nonzero(x > CL)
sk_L = N_L / (N_L + N_U + 1)
sk_U = N_U / (N_L + N_U + 1)
LB = CL - 1/sk_U * (CL - x.min())
UB = CL + 1/sk_L * (x.max() - CL)
CL2 = np.median(x)
S_x = np.sum((x - x.mean())**2) / (x.shape[0] - 1)
#print(f"S :: {S_x} | var :: {x.var()}")
print(f"{np.sqrt(-2*S_x/N_L * np.log(10**-20))} || {(CL - x.min())}")
LB2 = CL2 - sk_L * np.sqrt(-2*S_x/N_L * np.log(10**-20))
UB2 = CL2 + sk_U * np.sqrt(-2*S_x/N_U * np.log(10**-20))
sns.stripplot(x=x)
plt.scatter(CL, 0, marker='o', c='r')
plt.axvline(LB, c='r', label='from PSO')
```

```
plt.axvline(UB, c='r')
plt.scatter(CL2, 0, marker='o', c='g')
plt.axvline(LB2, c='g', label='from DE')
plt.axvline(UB2, c='g')
plt.title(f"Size of x :: {x.shape[0]}")
plt.legend()
plt.show()
```

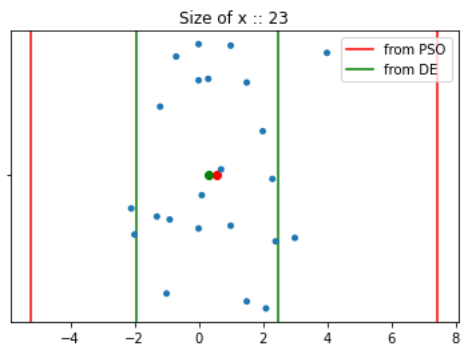
7.650518534711755 || 2.4222222222222225



5.882748794601619 || 3.0357142857142856

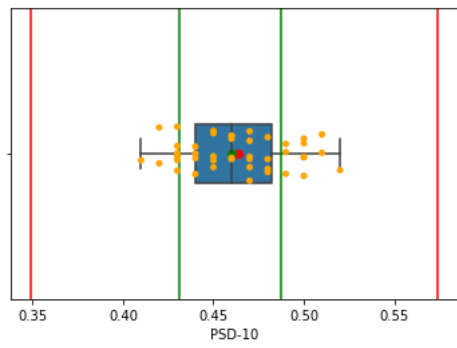


4.512926064587707 || 2.6521739130434785

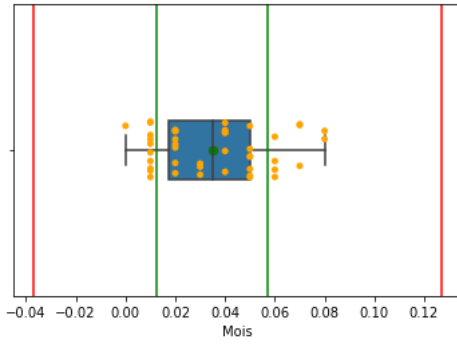


```
In [ ]: for attr in columns_x:
    sns.boxplot(x=attr, data=data_train, width=.2)
    sns.stripplot(x=attr, data=data_train, color='orange')
    x = data_train[attr]
    CL = x.mean()
    N_L = np.count_nonzero(x < CL)
    N_U = np.count_nonzero(x > CL)
    sk_L = N_L / (N_L + N_U + 1)
    sk_U = N_U / (N_L + N_U + 1)
    LB = CL - 1/sk_U * (CL - x.min())
    UB = CL + 1/sk_L * (x.max() - CL)
    CL2 = x.median()
    S_x = np.sum((x - x.mean())**2) / (x.shape[0] - 1)
    #print(f"S :: {S_x} | var :: {x.var()}")
    print(f"{np.sqrt(-2*S_x/N_L * np.log(10**-20))} || {(CL - x.min())}")
    LB2 = CL2 - sk_L * np.sqrt(-2*S_x/N_L * np.log(10**-20))
    UB2 = CL2 + sk_U * np.sqrt(-2*S_x/N_U * np.log(10**-20))
    plt.scatter(CL, 0, marker='o', c='r')
    plt.axvline(LB, c='r')
    plt.axvline(UB, c='r')
    plt.scatter(CL2, 0, marker='o', c='g')
    plt.axvline(LB2, c='g')
    plt.axvline(UB2, c='g')
    plt.show()
```

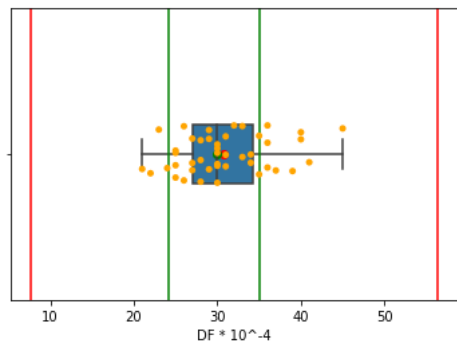
0.05642617139797006 || 0.05340909090909102



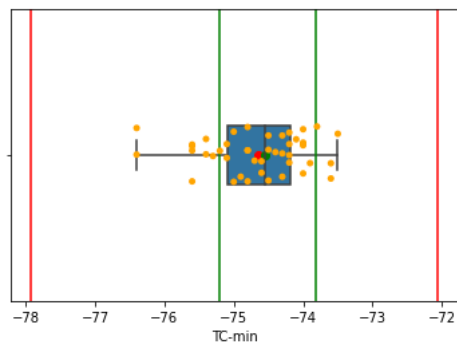
0.045536421931445005 || 0.035227272727272746



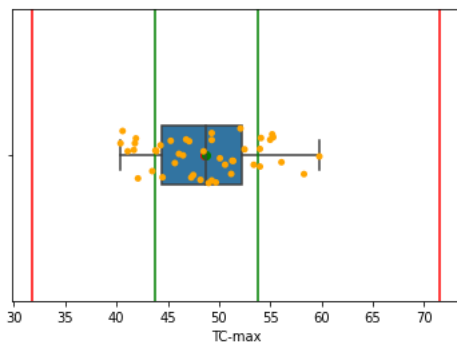
10.43225537277612 || 9.75



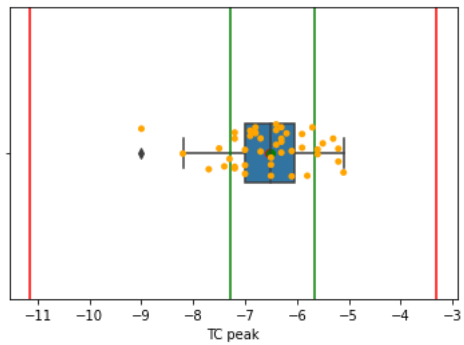
1.4971447668453965 || 1.7454545454545496



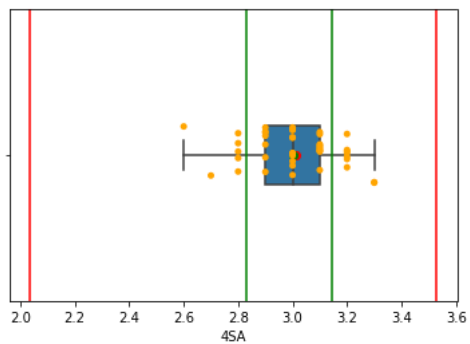
10.29522556415992 || 8.22727272727272



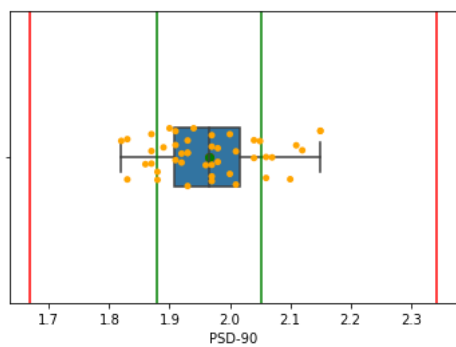
1.7379095453858913 || 2.463636363636362



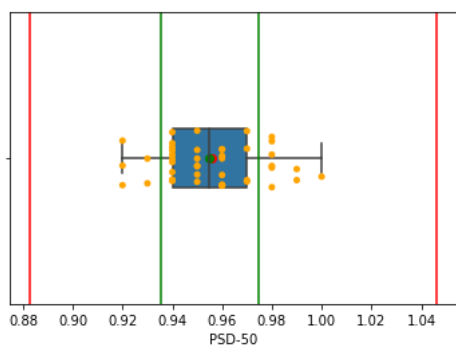
0.3030359602794594 || 0.4136363636363636



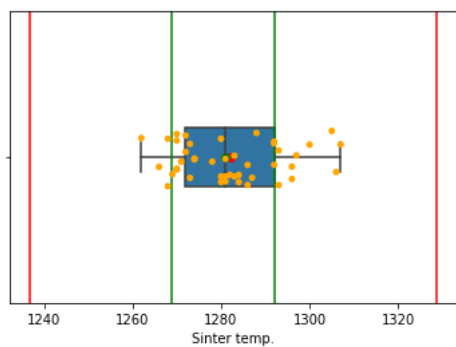
0.17785480734997847 || 0.1452272727272721



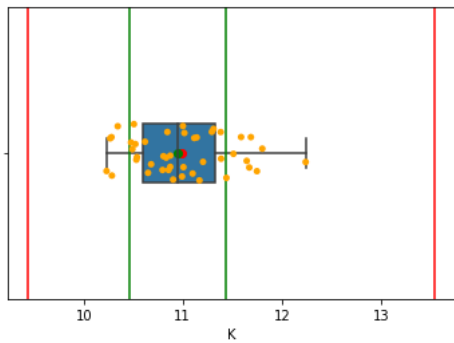
0.0402980821085836 || 0.03568181818181804



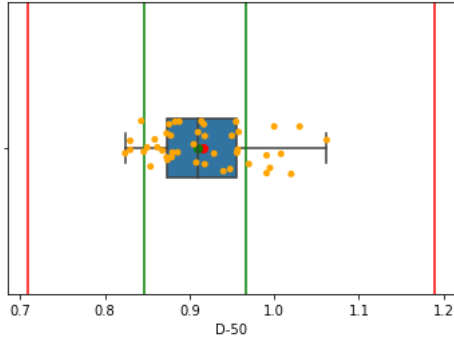
22.65309896088695 || 20.15909090909099



0.9900452577047842 || 0.7617272727272706



0.11756897442916703 || 0.09204545454545454



Normalize

```
In [ ]: data_test = data_test.copy()
data_train = data_train.copy()
data_test.loc(axis=1)[columns_y] = data_test["RK"].apply(lambda x: (x - data["RK"].min())/(data["RK"].max() - data["RK"].min()))
data_train.loc(axis=1)[columns_y] = data_train["RK"].apply(lambda x: (x - data["RK"].min())/(data["RK"].max() - data["RK"].min()))
```

RF

```
In [ ]: number_new_samples = 100
desired_fitness = 10**-6
RF = RandomForestRegressor(n_estimators=100, max_features=3, max_depth=3)
RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
```

Out [42]: RandomForestRegressor(max_depth=3, max_features=3)

```
In [ ]: ##### Sanity check
test_prediction = RF.predict(data_test.loc(axis=1)[columns_x])
train_prediction = RF.predict(data_train.loc(axis=1)[columns_x])
print(f"np.array(data_test.loc(axis=1)[columns_y]) || {test_prediction}")
print(f"MSE :: {mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction)}")
print(f"RMSE :: {np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction))}")
print(f"MAPE :: {mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction)}")
print(f"MSE :: {mean_squared_error(data_train.loc(axis=1)[columns_y], train_prediction)}")
print(f"RMSE :: {np.sqrt(mean_squared_error(data_train.loc(axis=1)[columns_y], train_prediction))}")
print(f"MAPE :: {mean_absolute_percentage_error(data_train.loc(axis=1)[columns_y], train_prediction)}")
```

```
[[17.687]
 [17.388]
 [20.23 ]
 [20.164]
 [17.112]
 [20.156]
 [16.606]
 [17.577]
 [16.681]
 [16.64 ]
 [17.286]
 [17.187]
 [18.092]
 [18.527]] || [17.59630298 18.35669677 18.45368808 17.95078283 16.86860633 18.19208557
 17.85404484 17.15536499 18.18785281 18.3228141 17.57528093 18.07916036
 17.27759892 17.79490564]
MSE :: 1.5595085386399663
RMSE :: 1.2488028421812494
MAPE :: 0.05813187083296282
MSE :: 0.20030178423337874
RMSE :: 0.4475508733466831
MAPE :: 0.021342087066341872
```

KNeighbors

```
In [ ]: number_new_samples = 100
desired_fitness = 10**-6
```

```
RF = KNeighborsRegressor()
RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
```

Out [31]: KNeighborsRegressor()

```
In [ ]: ##### Sanity check
test_prediction = RF.predict(data_test.loc(axis=1)[columns_x])
print(f"{np.array(data_test.loc(axis=1)[columns_y])} || {test_prediction}")
print(f"MSE :: {mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction)}")
print(f"RMSE :: {np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction))}")
print(f"MAPE :: {mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction)}")
```

```
[[17.687]
 [17.388]
 [20.23 ]
 [20.164]
 [17.112]
 [20.156]
 [16.606]
 [17.577]
 [16.681]
 [16.64 ]
 [17.286]
 [17.187]
 [18.092]
 [18.527]] || [16.9108 17.4252 17.4634 17.946 17.9188 17.4108 17.7914 17.7914 17.9876
 17.6956 18.2972 17.5622 17.8562 17.7914]
MSE :: 1.9569402885714282
RMSE :: 1.39890681911167804
MAPE :: 0.059609993216756085
```

DE

```
In [ ]: RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
##### DE
verbose = False
m = data_train.shape[0]/10
CL = data_train.loc(axis=1)[columns_x].median() #np.mean(data_train.loc(axis=1)[columns_x])
N_L = np.count_nonzero(data_train.loc(axis=1)[columns_x] < CL, axis=0)
N_U = np.count_nonzero(data_train.loc(axis=1)[columns_x] > CL, axis=0)
print(f"N_L :: {N_L} | N_U :: {N_U}")
LS = N_L / (N_L + N_U + m)
RS = N_U / (N_L + N_U + m)
S_x = np.sum((data_train.loc(axis=1)[columns_x] - CL)**2, axis=0) / (data_train.shape[0]-1)
#print(S_x)
LB = CL - LS * np.sqrt(-2*S_x/N_L * np.log(10**-20))
UB = CL + RS * np.sqrt(-2*S_x/N_U * np.log(10**-20))
#LB = np.maximum(LB, 0)
LB = np.minimum(LB, np.min(data_train.loc(axis=1)[columns_x], axis=0))
UB = np.maximum(UB, np.max(data_train.loc(axis=1)[columns_x], axis=0))
print(f"UB (from DE) :\n{UB}")
print(f"LB (from DE) :\n{LB}")
# DE
def f(x):
    if not LB.equals(np.minimum(LB, x)):
        return np.inf
    if not UB.equals(np.maximum(UB, x)):
        return np.inf
    y = data_train['RK']
    if verbose: print(f"x :: {x}")
    return np.min( np.abs((y - RF.predict(np.array(x).reshape(1, -1))) / y))

# def f(x):
#     n = data_train.shape[1]
#     #print(1/n * np.sum(np.min(np.absolute(data_train - x))))
#     return 1/n * np.sum(np.min(np.absolute(data_train - x))) #np.min(1/n * np.sum(np.absolute(data_train - x), axis=1))

print(f"Init:")
M = number_new_samples # number of data to produce
data_new = pd.DataFrame(columns=columns_x)
for i in tqdm(range(M)):
    new_x = []
    for attr in columns_x:
        new_x.append(LB[attr] + np.random.uniform() * (UB[attr] - LB[attr]))
    new_data = pd.DataFrame(np.array(new_x).reshape(1, -1), columns=columns_x)
    data_new = data_new.append(new_data, ignore_index=True)
if verbose: print(data_new.head())
print(f"Mutation, Crossover and Selection:")
count = 0
for g in tqdm(range(50)): # iterations
    fitness = 0
    for i in range(data_new.shape[0]):
        # Mutation
        F = 1.2#0.6 # scaling factor between [0,2]
        parents = data_new.loc[np.random.randint(0, data_new.shape[0], 3), :]
        p1 = parents.iloc[0]
        p2 = parents.iloc[1]
        p3 = parents.iloc[2]
        h = p1 + F * (p2 - p3)
        # Crossover
```

```

P_cr = 0.7 # probability of crossover
v_x = []
for attr in columns_x:
    v_x.append(h[attr] if np.random.uniform() <= P_cr else data_new.loc[i, attr])
v = pd.DataFrame(np.array(v_x).reshape(1, -1), columns=columns_x)
# Selection
if f(v.loc[0, :]) < f(data_new.loc[i, :]):
    for attr in columns_x:
        data_new.at[i, attr] = v[attr]
    fitness += f(data_new.loc[i, :])
if fitness < 10**-3:
    break

data_new['RK'] = RF.predict(np.array(data_new[columns_x]))
data_new

```

```

N_L :: [18 22 19 22 22 20 14 22 22 20 22 22] | N_U :: [21 22 19 22 22 21 19 22 22 21 22 22]
UB (from DE) :
PSD-10      0.520
Mois        0.080
DF * 10^-4  45.000
TC-min      -73.500
TC-max      59.800
TC peak     -5.100
4SA         3.300
PSD-90      2.150
PSD-50      1.000
Sinter temp. 1307.000
K           12.242
D-50        1.062
dtype: float64
LB (from DE) :
PSD-10      0.410
Mois        0.000
DF * 10^-4  21.000
TC-min      -76.400
TC-max      40.400
TC peak     -9.000
4SA         2.600
PSD-90      1.820
PSD-50      0.920
Sinter temp. 1262.000
K           10.232
D-50        0.824
dtype: float64
Init:

HBox(children=(FloatProgress(value=0.0), HTML(value='')))

Mutation, Crossover and Selection:

HBox(children=(FloatProgress(value=0.0, max=50.0), HTML(value='')))

```

Out [20]:

	PSD-10	Mois	DF * 10^-4	TC-min	TC-max	TC peak	4SA	PSD-90	PSD-50	Sinter temp.	K	D-50	RK
0	0.424534	0.050992	34.193642	-74.974889	54.772041	-5.516138	2.677012	2.125325	0.989495	1299.386510	10.555182	0.970910	19.05609
1	0.493057	0.039038	42.335964	-75.149105	42.563486	-8.818178	3.140259	1.924202	0.930890	1270.082554	11.295133	0.977993	17.22843
2	0.492869	0.041976	23.231254	-74.194701	44.884660	-8.707304	3.174173	1.850270	0.946540	1294.282454	10.567126	1.013649	17.33350
3	0.498169	0.008329	39.210334	-74.032555	40.611605	-6.830759	3.299804	2.057312	0.922951	1301.917117	10.285577	0.890289	17.45028
4	0.422867	0.047971	31.488215	-74.728455	59.342253	-5.525617	2.716240	2.108030	0.933034	1293.822625	10.518382	0.920085	18.82622
...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	0.492643	0.060404	34.040166	-74.424247	50.859805	-5.980453	3.209610	2.071893	0.941531	1270.780250	10.536896	0.999998	17.43992
96	0.424198	0.017351	35.249649	-75.840769	49.649650	-6.427759	2.756225	2.125031	0.953894	1303.751438	11.023909	0.890611	17.63325
97	0.412809	0.019139	22.155353	-75.473789	43.375067	-5.216630	2.610301	1.984183	0.929866	1268.628807	12.131726	0.991707	17.41680
98	0.453468	0.040604	23.449163	-75.075146	57.718348	-5.595344	3.061506	1.880323	0.939906	1284.937690	12.207327	0.905242	18.95461
99	0.420965	0.043647	34.830536	-76.047275	49.340489	-5.774944	2.723614	1.955419	0.946588	1270.992802	11.768882	1.032851	17.67109

100 rows × 13 columns

```

In [ ]: data_new_train = data_train.append(data_new)
data_new_train['Type'] = np.concatenate((np.full((data_train.shape[0]), "Raw samples"), np.full((data_new.shape[0]), "DEVSC
#TODO

```

```

In [ ]: sns.pairplot(data_new_train, hue='Type')

```

Out [22]: <seaborn.axisgrid.PairGrid at 0x7fbabf7e6d90>



PSO

```
In [ ]: RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
##### PSO
# Domain expansion
CL = np.mean(data_train.loc(axis=1)[columns_x])
N_L = np.count_nonzero(data_train.loc(axis=1)[columns_x] < CL, axis=0)
N_U = np.count_nonzero(data_train.loc(axis=1)[columns_x] > CL, axis=0)
s_p = 1
sk_L = N_L / (N_L + N_U + s_p)
sk_U = N_U / (N_L + N_U + s_p)
#print(f"sk_L :: {sk_L}")
LB = CL - 1/sk_U * (CL - np.min(data_train.loc(axis=1)[columns_x], axis=0))
UB = CL + 1/sk_L * (np.max(data_train.loc(axis=1)[columns_x], axis=0) - CL)
#print(f"sk_U :: {sk_U}")
#LB = np.maximum(LB, 0)
print(f"UB :\n{UB}")
print(f"LB :\n{LB}")
# This domain expansion is better
# m = data_train.shape[0]/10
# CL = data_train.loc(axis=1)[columns_x].median() #np.mean(data_train.loc(axis=1)[columns_x])
# N_L = np.count_nonzero(data_train.loc(axis=1)[columns_x] < CL, axis=0)
# N_U = np.count_nonzero(data_train.loc(axis=1)[columns_x] > CL, axis=0)
# print(f"N_L :: {N_L} | N_U :: {N_U}")
# LS = N_L / (N_L + N_U + m)
# RS = N_U / (N_L + N_U + m)
# S_x = np.sum((data_train.loc(axis=1)[columns_x] - CL)**2, axis=0) / (data_train.shape[0]-1)
# #print(S_x)
# LB = CL - LS * np.sqrt(-2*S_x/N_L * np.log(10**-20))
# UB = CL + RS * np.sqrt(-2*S_x/N_U * np.log(10**-20))
# #LB = np.maximum(LB, 0)
# LB = np.minimum(LB, np.min(data_train.loc(axis=1)[columns_x], axis=0))
# UB = np.maximum(UB, np.max(data_train.loc(axis=1)[columns_x], axis=0))

# PSO
def f(x):
```

```

verbose = False
#y = x[:, -1]
y = data_train['RK']
if verbose:
    print(f"x :: {x}")
cost = np.zeros((x.shape[0]))
c = 0
for i in range(x.shape[0]):
    cost[i] = np.min(100 * np.abs((y - RF.predict(x[i, :].reshape(1, -1))) / y))

return cost

bounds = np.array([[a,b,c,d,e,f,g,h,i,j,k,l] for (a,b,c,d,e,f,g,h,i,j,k,l) in itertools.product(UB.append(LB)['PSD-10'], UB
    UB.append(LB)['TC-min'], UB.append(LB)['TC-max'], UB.append(LB)['TC peak'], UB.append(LB)['4SA'], UB.app
    UB.append(LB)['PSD-50'], UB.append(LB)['Sinter temp.'], UB.append(LB)['K'], UB.append(LB)['D-50'])])

optimizer = ps.single.GlobalBestPSO(n_particles=50, dimensions=columns_x.shape[0], options={'c1': 2, 'c2': 2, 'w': 0.9}, \
    #init_pos=bounds[np.random.choice(bounds.shape[0], size=50, replace=False)], \
    bounds=(np.array(LB), np.array(UB)), \
    ftol=10**-6, \
    ftol_iter=50, \
    bh_strategy='nearest', \
    oh_strategy={'w': "lin_variation"}, \
    )
cost, pos = optimizer.optimize(f, iters=500, n_processes=4, verbose=False)
plot_cost_history(optimizer.cost_history)
plt.title("cost of 1st sample")
plt.show()
# # Make animation
# animation = plot_contour(pos_history=optimizer.pos_history, \
#                             designer=Designer(limits=[(LB['Mois'], UB['Mois']), (LB['X2'], UB['X2'])]), label=["Mois", "X2"])
#                             animator=Animator(repeat_delay=5000), \
#                             mark=pos, \
#                             )
# # Enables us to view it in a Jupyter notebook
# animation.save('plot0.gif', writer='imagemagick', fps=10)
# Image(url='plot0.gif')
#print(f"new sample :: {pos}")
data_new = pd.DataFrame(pos.reshape(1, -1), columns=columns_x)
data_new['RK'] = RF.predict(pos.reshape(1, -1))
data_new
for i in tqdm(range(1, number_new_samples)):
    optimizer = ps.single.GlobalBestPSO(n_particles=50, dimensions=columns_x.shape[0], options={'c1': 2, 'c2': 2, 'w': 0.9},
        #init_pos=bounds[np.random.choice(bounds.shape[0], size=50, replace=False)], \
        bounds=(np.array(LB), np.array(UB)), \
        ftol=10**-6, \
        ftol_iter=50, \
        bh_strategy='nearest', \
        oh_strategy={'w': "lin_variation"}, \
        )# https://github.com/ljvmiranda921/pyswarms/blob/master/docs/examples/tutorials/option
    cost, pos = optimizer.optimize(f, iters=500, n_processes=4, verbose=False)
    nd = pd.DataFrame(pos.reshape(1, -1), columns=columns_x)
    nd['RK'] = RF.predict(pos.reshape(1, -1))
    data_new = data_new.append(nd, ignore_index=True)

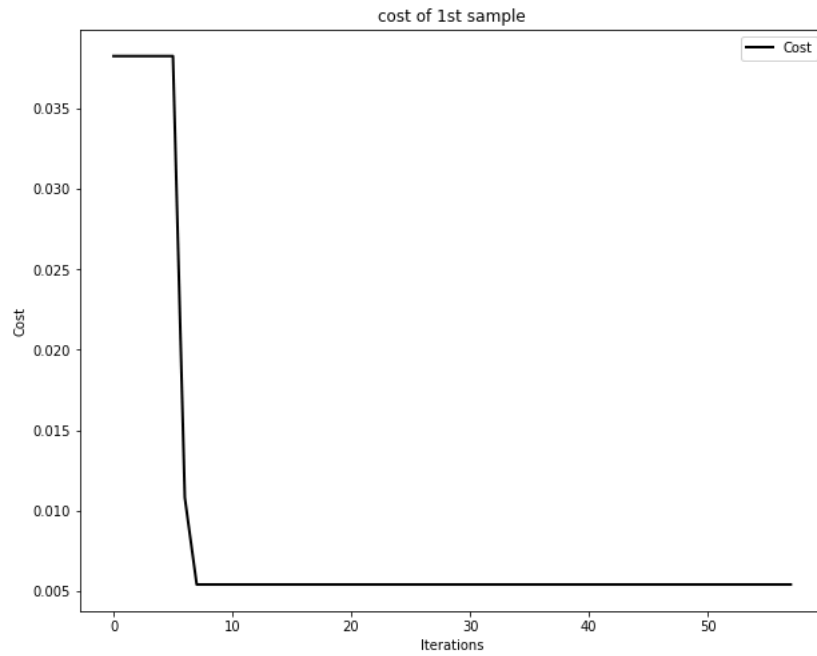
data_new

```

```

UB :
PSD-10      0.557053
Mois        0.117882
DF * 10^-4  48.294444
TC-min      -72.087692
TC-max      70.437500
TC peak     -3.994902
4SA         3.519608
PSD-90      2.403846
PSD-50      1.019098
Sinter temp. 1329.906250
K           13.341055
D-50        1.124688
dtype: float64
LB :
PSD-10      0.316667
Mois        -0.023231
DF * 10^-4  7.908333
TC-min      -77.948235
TC-max      32.170952
TC peak     -12.235385
4SA         2.476923
PSD-90      1.699216
PSD-50      0.872000
Sinter temp. 1245.883333
K           9.296785
D-50        0.721595
dtype: float64

```



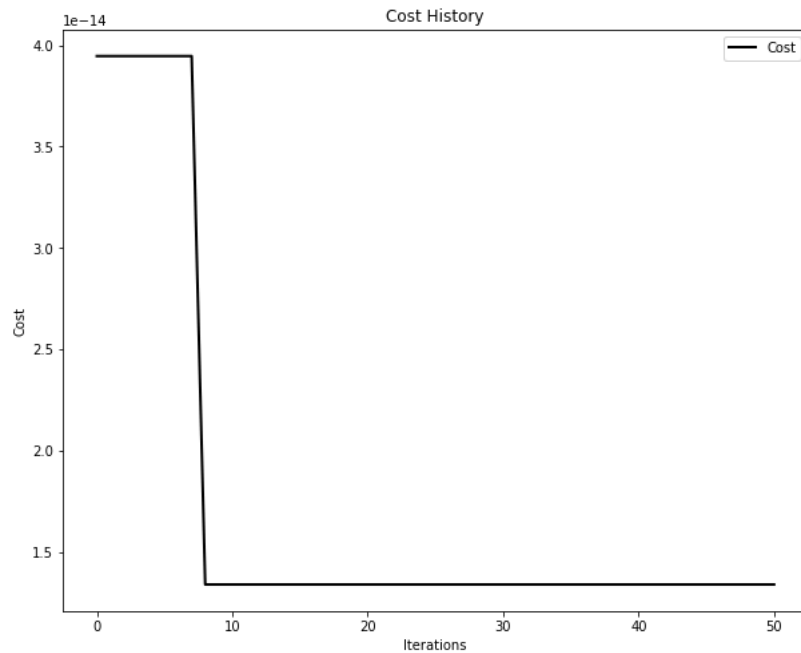
HBox(children=(FloatProgress(value=0.0, max=99.0), HTML(value='')))

Out [21]:

	PSD-10	Mois	DF * 10 ⁻⁴	TC-min	TC-max	TC peak	4SA	PSD-90	PSD-50	Sinter temp.	K	D-50	RK
0	0.316667	0.117882	7908333	-74.581579	67.872622	-12.235385	3.459041	2.403846	0.872000	1325.841153	10.501710	1.002251	0.368085
1	0.557053	0.091616	34.913176	-72.087692	70.437500	-6.302101	2.476923	1.699216	1.004006	1295.550994	13.324679	0.721595	0.680453
2	0.316667	0.117882	27948548	-77.367641	32.170952	-11.803675	2.825586	2.281318	1.019098	1287.623287	9.296785	1.124688	0.344800
3	0.317815	0.010126	13.767551	-72.818583	33.359103	-7.800459	3.519608	1.790529	1.019098	1329.906250	9.296785	1.118686	0.418970
4	0.557053	0.117882	7908333	-74.404906	70.437500	-11.199316	3.519608	1.969984	1.019098	1329.906250	13.291550	0.721595	0.484013
...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	0.557053	0.113232	39.679369	-74.127634	70.437500	-11.367415	2.632204	1.699216	1.019098	1245.883333	11.453956	0.941270	0.599920
96	0.454088	0.061942	18.007599	-77.648928	44.903677	-6.448374	3.472918	2.403846	0.875763	1282.322411	9.366662	0.881660	0.386538
97	0.316667	0.098431	15.790154	-75.067415	44.559455	-10.401305	2.543652	1.947124	0.949014	1309.940751	13.064817	0.830037	0.368065
98	0.557053	-0.023231	48.294444	-77.694321	52.054692	-9.163700	2.737689	1.699216	1.019098	1245.969279	13.341055	0.818074	0.592543
99	0.351628	0.032778	14.296257	-73.852919	64.876841	-6.963238	2.668711	2.340242	0.872000	1321.034037	12.292390	1.124688	0.414794

100 rows × 13 columns

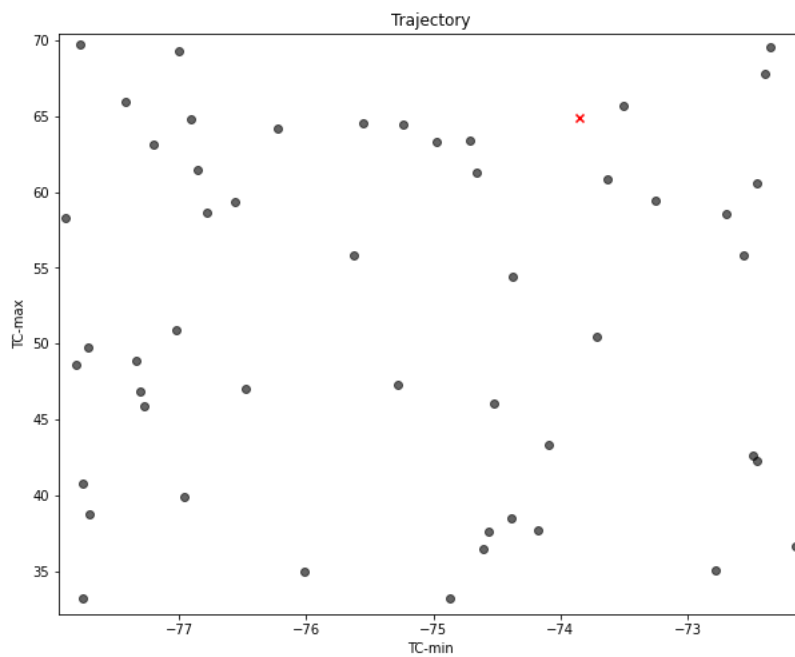
```
In [ ]: plot_cost_history(optimizer.cost_history)
plt.show()
print(data_new.iloc[-1])
# Make animation
animation = plot_contour(pos_history=np.array(optimizer.pos_history)[:,:3:5], \
                        designer=Designer(limits=[(LB['TC-min'], UB['TC-min']), (LB['TC-max'], UB['TC-max'])], label=["TC-
animator=Animator(repeat_delay=5000), \
mark=data_new.iloc[-1].loc[['TC-min', 'TC-max']], \
)
# Enables us to view it in a Jupyter notebook
animation.save('plot1.gif', writer='imagemagick', fps=1)
Image(url='plot1.gif')
```



2021-04-07 08:13:31,318 - matplotlib.animation - INFO - Animation.save using <class 'matplotlib.animation.ImageMagickWriter'>
 2021-04-07 08:13:31,322 - matplotlib.animation - INFO - MovieWriter._run: running command: convert -size 720x576 -depth 8 -delay 100.0 -loop 0 rgba:-plot1.gif

```
PSD-10      0.351628
Mois        0.032778
DF * 10^-4  14.296257
TC-min      -73.852919
TC-max      64.876841
TC peak    -6.963238
4SA         2.668711
PSD-90     2.340242
PSD-50     0.872000
Sinter temp. 1321.034037
K           12.292390
D-50       1.124688
RK          0.414794
Name: 99, dtype: float64
```

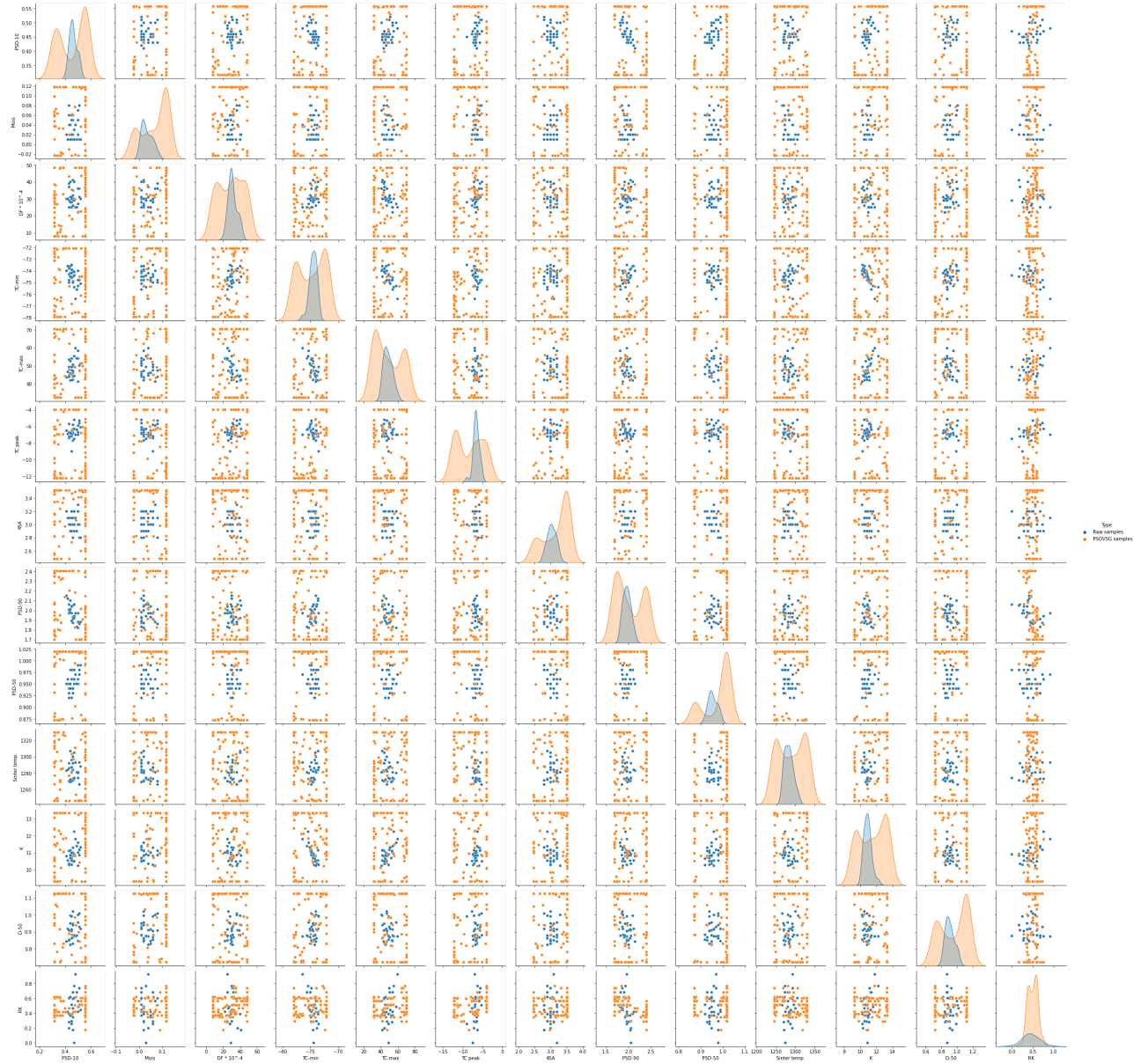
Out [22]:



```
In [ ]: data_new_train = data_train.append(data_new)
data_new_train['Type'] = np.concatenate((np.full((data_train.shape[0]), "Raw samples"), np.full((data_new.shape[0]), "PSOVS")),
#TODO
```

```
In [ ]: sns.pairplot(data_new_train, hue='Type')
```

Out [24]: <seaborn.axisgrid.PairGrid at 0x7febe633cd50>



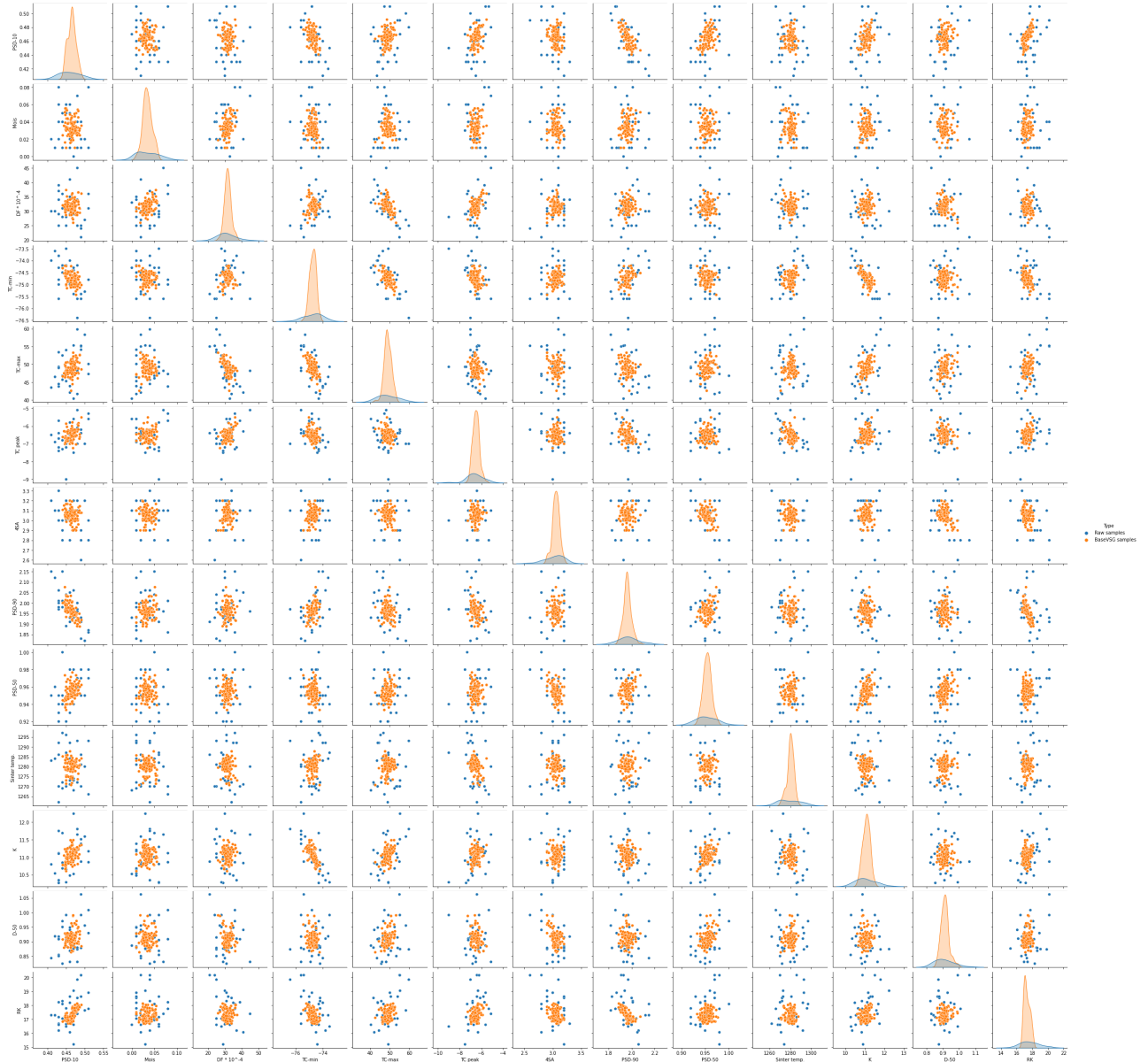
Middle Points

```
In [ ]: RF.fit(data_train.loc(axis=1)[columns_x], data_train["RK"])
data_new = data_train.copy()
for i in tqdm(range(number_new_samples)):
    index = np.random.choice(data_new.index, size=3, replace=False)
    nd = data_new.loc[index, columns_x].mean()
    nd = pd.DataFrame(np.array(nd).reshape(1, -1), columns=columns_x)
    nd['RK'] = RF.predict(nd)
    data_new = data_new.append(nd, ignore_index=True)
```

```
In [ ]: data_new_train = data_new
data_new_train['Type'] = np.concatenate((np.full((data_train.shape[0]), "Raw samples"), np.full((number_new_samples), "Base
sns.pairplot(data_new_train, hue='Type')
```

2021-03-23 15:50:09,705 - numexpr.utils - INFO - NumExpr defaulting to 2 threads.

Out [8]: <seaborn.axisgrid.PairGrid at 0x7f3e8fc3e690>



Prediction

```
In [ ]: data_train_vsg = data_train.append(data_new)
RF.fit(data_train_vsg.loc(axis=1)[columns_x], data_train_vsg["RK"])
test_prediction_vsg = RF.predict(data_test.loc(axis=1)[columns_x])
print(f"{np.array(data_test.loc(axis=1)[columns_y])} || {test_prediction_vsg}")
print(f"MSE :: {mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_vsg)}")
print(f"RMSE :: {np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_vsg))}")
print(f"MAPE :: {mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_vsg)}")
```

```
[[0.49433287]
[0.43487771]
[1.         ]
[0.98687612]
[0.37999602]
[0.98528534]
[0.2793796 ]
[0.47245973]
[0.2942931 ]
[0.28614039]
[0.41459535]
[0.39490952]
[0.57486578]
[0.66136409]] || [[0.49773315 0.6668105 0.73048519 0.59390336 0.31541857 0.66699145
0.61630543 0.32507457 0.62461722 0.66281567 0.4172579 0.57411016
0.39254325 0.60727779]
MSE :: 0.060062710058046066
RMSE :: 0.24507694721871753
MAPE :: 0.46548642547156366
```

AveMAPE

```
In [ ]: ave_mape_raw = []
ave_mape_psovsg = []
ave_mape_basevsg = []
ave_mape_devsg = []
ave_rmse_raw = []
ave_rmse_psovsg = []
ave_rmse_basevsg = []
ave_rmse_devsg = []
```

```

mape_raw = []
mape_psovs = []
mape_basevs = []
mape_devs = []
rmse_raw = []
rmse_psovs = []
rmse_basevs = []
rmse_devs = []
nbr_y_zero = []
PSO = True
base = True
DE = True
normalize = False
n_runs = 15
train_sizes = [25, 30]#[5, 10, 15, 20, 25, 30]
for train_size in train_sizes:
    print(f"train size :: {train_size}")
    offset = (train_size // 5) - 1
    avemape_raw = 0
    avemape_psovs = 0
    avemape_basevs = 0
    avemape_devs = 0
    avermse_raw = 0
    avermse_psovs = 0
    avermse_basevs = 0
    avermse_devs = 0
    mape_raw.append([])
    mape_psovs.append([])
    mape_basevs.append([])
    mape_devs.append([])
    rmse_raw.append([])
    rmse_psovs.append([])
    rmse_basevs.append([])
    rmse_devs.append([])
    nbr_y_zero.append([])
    for run in tqdm(range(n_runs)):
        data = pd.read_csv("/content/drive/MyDrive/pso_data.csv", decimal=',', index_col=0)
        columns_x = data.columns[data.columns != 'RK']
        columns_y = data.columns[data.columns == 'RK']
        data = data.apply(pd.to_numeric)
        data['TC-min'] = - data['TC-min']
        data['TC peak'] = - data['TC peak']
        data_train, data_test = train_test_split(data, train_size=train_size, random_state=run)
        if normalize:
            data_test = data_test.copy()
            data_train = data_train.copy()
            data_test.loc[:, columns_y] = data_test["RK"].apply(lambda x: (x - data["RK"].min())/(data["RK"].max() - data["RK"].min()))
            data_train.loc[:, columns_y] = data_train["RK"].apply(lambda x: (x - data["RK"].min())/(data["RK"].max() - data["RK"].min()))
            nbr_y_zero[offset].append(np.count_nonzero(np.isclose(data_test.loc[:, columns_y], np.zeros(data_test.shape))))
        #####
        number_new_samples = 100
        desired_fitness = 10**-6
        RF = RandomForestRegressor(n_estimators=10)
        RF = KNeighborsRegressor()
        RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
        test_prediction = RF.predict(data_test.loc(axis=1)[columns_x])
        avermse_raw += np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction))
        avemape_raw += mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction)
        mape_raw[offset].append(mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction))
        rmse_raw[offset].append(np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction)))
        #####
        RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
        if PSO:
            ##### PSO
            # Domain expansion
            CL = np.mean(data_train.loc(axis=1)[columns_x])
            N_L = np.count_nonzero(data_train.loc(axis=1)[columns_x] < CL, axis=0)
            N_U = np.count_nonzero(data_train.loc(axis=1)[columns_x] > CL, axis=0)
            s_p = 1
            sk_L = N_L / (N_L + N_U + s_p)
            sk_U = N_U / (N_L + N_U + s_p)
            #print(f"sk_L :: {sk_L}")
            LB = CL - 1/sk_U * (CL - np.min(data_train.loc(axis=1)[columns_x], axis=0))
            UB = CL + 1/sk_L * (np.max(data_train.loc(axis=1)[columns_x], axis=0) - CL)
            #print(f"sk_U :: {sk_U}")
            #LB = np.maximum(LB, 0)
            # This domain expansion is better
            # m = data_train.shape[0]/10
            # CL = data_train.loc(axis=1)[columns_x].median() #np.mean(data_train.loc(axis=1)[columns_x])
            # N_L = np.count_nonzero(data_train.loc(axis=1)[columns_x] < CL, axis=0)
            # N_U = np.count_nonzero(data_train.loc(axis=1)[columns_x] > CL, axis=0)
            # print(f"N_L :: {N_L} | N_U :: {N_U}")
            # LS = N_L / (N_L + N_U + m)
            # RS = N_U / (N_L + N_U + m)
            # S_x = np.sum((data_train.loc(axis=1)[columns_x] - CL)**2, axis=0) / (data_train.shape[0]-1)
            # #print(S_x)

```

```

# LB = CL - LS * np.sqrt(-2*S_x/N_L * np.log(10**-20))
# UB = CL + RS * np.sqrt(-2*S_x/N_U * np.log(10**-20))
# #LB = np.maximum(LB, 0)
# LB = np.minimum(LB, np.min(data_train.loc(axis=1)[columns_x], axis=0))
# UB = np.maximum(UB, np.max(data_train.loc(axis=1)[columns_x], axis=0))

# PSO
def f(x):
    verbose = False
    #y = x[:, -1]
    y = data_train['RK']
    if verbose:
        print(f"x :: {x}")
    cost = np.zeros((x.shape[0]))
    c = 0
    for i in range(x.shape[0]):
        cost[i] = np.min(100 * np.abs((y - RF.predict(x[i, :].reshape(1, -1)))))
        # if cost[i] > 0.1:
        #     cost[i] = np.inf

    return cost

bounds = np.array([[a,b,c,d,e,f,g,h,i,j,k,l] for (a,b,c,d,e,f,g,h,i,j,k,l) in itertools.product(UB.append(LB)['PSD-10
    UB.append(LB)['TC-min'], UB.append(LB)['TC-max'], UB.append(LB)['TC peak'], UB.append(LB)['4SA'], U
    UB.append(LB)['PSD-50'], UB.append(LB)['Sinter temp.'], UB.append(LB)['K'], UB.append(LB)['D-50'])])

optimizer = ps.single.GlobalBestPSO(n_particles=50, dimensions=columns_x.shape[0], options={'c1': 2, 'c2': 2, 'w': 0.
    #init_pos=bounds[np.random.choice(bounds.shape[0], size=50, replace=False)], \
    bounds=(np.array(LB), np.array(UB)), \
    ftol=10**-6, \
    ftol_iter=50, \
    bh_strategy='nearest', \
    oh_strategy={'w': "lin_variation"}, \
    )

cost, pos = optimizer.optimize(f, iters=500, n_processes=4, verbose=False)
data_new = pd.DataFrame(pos.reshape(1, -1), columns=columns_x)
data_new['RK'] = RF.predict(pos.reshape(1, -1))
data_new
for i in range(1, number_new_samples):
    optimizer = ps.single.GlobalBestPSO(n_particles=50, dimensions=columns_x.shape[0], options={'c1': 2, 'c2': 2, 'w':
        #init_pos=bounds[np.random.choice(bounds.shape[0], size=50, replace=False)], \
        bounds=(np.array(LB), np.array(UB)), \
        ftol=10**-6, \
        ftol_iter=50, \
        bh_strategy='nearest', \
        oh_strategy={'w': "lin_variation"}, \
        )# https://github.com/ljvmiranda921/pyswarms/blob/master/docs/examples/tutorials/

    cost, pos = optimizer.optimize(f, iters=500, n_processes=4, verbose=False)
    nd = pd.DataFrame(pos.reshape(1, -1), columns=columns_x)
    nd['RK'] = RF.predict(pos.reshape(1, -1))
    data_new = data_new.append(nd, ignore_index=True)
#####
data_train_vsg = data_train.append(data_new)
RF.fit(data_train_vsg.loc(axis=1)[columns_x], data_train_vsg["RK"])
test_prediction_psovsg = RF.predict(data_test.loc(axis=1)[columns_x])
avermse_psovsg += np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_psovsg))
avemape_psovsg += mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_psovsg)
mape_psovsg[offset].append(mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_psovsg))
rmse_psovsg[offset].append(np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_psovsg)))
#####
if base:
    RF.fit(data_train.loc(axis=1)[columns_x], data_train["RK"])
    data_new = data_train.copy()
    for i in range(number_new_samples):
        index = np.random.choice(data_new.index, size=3, replace=False)
        nd = data_new.loc[index, columns_x].mean()
        nd = pd.DataFrame(np.array(nd).reshape(1, -1), columns=columns_x)
        nd['RK'] = RF.predict(nd)
        data_new = data_new.append(nd, ignore_index=True)

#####
data_train_vsg = data_new
RF.fit(data_train_vsg.loc(axis=1)[columns_x], data_train_vsg["RK"])
test_prediction_basevsg = RF.predict(data_test.loc(axis=1)[columns_x])
avermse_basevsg += np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_basevsg))
avemape_basevsg += mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_basevsg)
mape_basevsg[offset].append(mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_basevsg))
rmse_basevsg[offset].append(np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_basevsg)))
#####
if DE:
    RF.fit(data_train.loc(axis=1)[columns_x], data_train['RK'])
    ##### DE
    m = data_train.shape[0]/10
    CL = data_train.loc(axis=1)[columns_x].median() #np.mean(data_train.loc(axis=1)[columns_x])
    N_L = np.count_nonzero(data_train.loc(axis=1)[columns_x] < CL, axis=0)

```



```

N_L = np.maximum(N_L, [0.1]*columns_x.shape[0])
N_U = np.count_nonzero(data_train.loc(axis=1)[columns_x] > CL, axis=0)
N_U = np.maximum(N_U, [0.1]*columns_x.shape[0])
LS = N_L / (N_L + N_U + m)
RS = N_U / (N_L + N_U + m)
S_x = np.sum((data_train.loc(axis=1)[columns_x] - CL)**2, axis=0) / (data_train.shape[0]-1)
LB = CL - LS * np.sqrt(-2*S_x/N_L * np.log(10**-20))
UB = CL + RS * np.sqrt(-2*S_x/N_U * np.log(10**-20))
LB = np.minimum(LB, np.min(data_train.loc(axis=1)[columns_x], axis=0))
UB = np.maximum(UB, np.max(data_train.loc(axis=1)[columns_x], axis=0))
# DE
def f(x):
    if not LB.equals(np.minimum(LB, x)):
        return np.inf
    if not UB.equals(np.maximum(UB, x)):
        return np.inf
    y = data_train['RK']
    return np.min(100* np.abs((y - RF.predict(np.array(x).reshape(1, -1)))))

M = number_new_samples # number of data to produce
data_new = pd.DataFrame(columns=columns_x)
for i in range(M):
    new_x = []
    for attr in columns_x:
        new_x.append(LB[attr] + np.random.uniform() * (UB[attr] - LB[attr]))
    new_data = pd.DataFrame(np.array(new_x).reshape(1, -1), columns=columns_x)
    data_new = data_new.append(new_data, ignore_index=True)
count = 0
for g in range(50): # iterations
    fitness = 0
    for i in range(data_new.shape[0]):
        # Mutation
        F = 1.2#0.6 # scaling factor between [0,2]
        parents = data_new.loc[np.random.randint(0, data_new.shape[0], 3), :]
        p1 = parents.iloc[0]
        p2 = parents.iloc[1]
        p3 = parents.iloc[2]
        h = p1 + F * (p2 - p3)
        # Crossover
        P_cr = 0.7 # probability of crossover
        v_x = []
        for attr in columns_x:
            v_x.append(h[attr] if np.random.uniform() <= P_cr else data_new.loc[i, attr])
        v = pd.DataFrame(np.array(v_x).reshape(1, -1), columns=columns_x)
        # Selection
        if f(v.loc[0, :]) < f(data_new.loc[i, :]):
            for attr in columns_x:
                data_new.at[i, attr] = v[attr]
            fitness += f(data_new.loc[i, :])
        if fitness < 10**-3:
            break

data_new['RK'] = RF.predict(np.array(data_new[columns_x]))
#####
data_train_vsg = data_new
RF.fit(data_train_vsg.loc(axis=1)[columns_x], data_train_vsg["RK"])
test_prediction_devsg = RF.predict(data_test.loc(axis=1)[columns_x])
avermse_devsg += np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_devsg))
avemape_devsg += mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_devsg)
mape_devsg[offset].append(mean_absolute_percentage_error(data_test.loc(axis=1)[columns_y], test_prediction_devsg))
rmse_devsg[offset].append(np.sqrt(mean_squared_error(data_test.loc(axis=1)[columns_y], test_prediction_devsg)))

ave_mape_raw.append(avemape_raw / n_runs)
ave_mape_psovg.append(avemape_psovg / n_runs)
ave_mape_basevsg.append(avemape_basevsg / n_runs)
ave_mape_devsg.append(avemape_devsg / n_runs)
ave_rmse_raw.append(avermse_raw / n_runs)
ave_rmse_psovg.append(avermse_psovg / n_runs)
ave_rmse_basevsg.append(avermse_basevsg / n_runs)
ave_rmse_devsg.append(avermse_devsg / n_runs)

```

train size :: 25

HBox(children=(FloatProgress(value=0.0, max=15.0), HTML(value='')))

train size :: 30

HBox(children=(FloatProgress(value=0.0, max=15.0), HTML(value='')))

```

In [ ]: print(f"ave_mape_raw = {ave_mape_raw}")
print(f"ave_mape_psovg = {ave_mape_psovg}")
print(f"ave_mape_basevsg = {ave_mape_basevsg}")
print(f"ave_mape_devsg = {ave_mape_devsg}")
print(f"ave_rmse_raw = {ave_rmse_raw}")
print(f"ave_rmse_psovg = {ave_rmse_psovg}")
print(f"ave_rmse_basevsg = {ave_rmse_basevsg}")

```

```
ave_mape_raw = [0.05589848144166814, 0.051881610146054, 0.052100895895879094, 0.04977705711834957, 0.04881759864641714, 0.05206677529883453]
ave_mape_psovsog = [0.056215211053157, 0.05415252339636103, 0.05091767744855405, 0.04948175334053989, 0.04903607776568666, 0.05229790716982919]
ave_mape_baseavg = [0.05575623308922619, 0.05375756007429254, 0.05123699251070237, 0.049828356702321025, 0.04928185508362532, 0.05285916079147386]
ave_mape_devsg = [0.05589848144166814, 0.05333078525950121, 0.05156244792077824, 0.0508148007560965, 0.04911783632131943, 0.0525524684203634]
ave_rmse_raw = [1.289451221317378, 1.2297843600036122, 1.177173472032519, 1.121457426616207, 1.087828817717436, 1.135707393397693]
ave_rmse_psovsog = [1.2934659057531392, 1.229078346055249, 1.15768593147695, 1.113287946526232, 1.102792423626294, 1.1538734231789116]
ave_rmse_baseavg = [1.284750625996762, 1.2216507046572124, 1.192549943810593, 1.1360616749001358, 1.126742609418369, 1.1762465940838889]
ave_rmse_devsg = [1.289451221317378, 1.219062133806335, 1.1650770089358022, 1.147400736020279, 1.09729863534367, 1.163870638719559]
mape_raw = [[0.048561097469817543, 0.05754441262503065, 0.04994420547561992, 0.055613140586969905, 0.05318472336537725, 0.08083613818808265, 0.05381752279960205, 0.04871528082670679, 0.060081716967095376, 0.05001449038661386, 0.05060995969732827, 0.05357848247589113, 0.07943844043239895, 0.05196616063406209, 0.048871703151367074, 0.056011735929335544, 0.05034803487499846, 0.0560107652737417, 0.053268331282838064, 0.0790626386049986, 0.0517770791253405]
mape_devsg = [[0.048561097469817516, 0.05754441262503065, 0.04994420547561992, 0.055613140586969905, 0.05318472336537725, 0.08083613818808265, 0.053817522799602044, 0.048871703151367074, 0.056011735929335544, 0.05034803487499846, 0.0560107652737417, 0.053268331282838064, 0.0790626386049986, 0.0517770791253405]
rmse_raw = [[1.241500902650518, 1.1940904449702439, 1.2293317304900846, 1.375106364184427, 1.3119949816195466, 1.55918392559895, 1.1346802063964403, 1.3862129362649, 1.241500902650518, 1.2241701385857813, 1.2252392266588545, 1.3820687967588154, 1.3247428975523468, 1.527751394221985, 1.0662210390547908, 1.425411, 1.2447840851760976, 1.150576394325683, 1.225343414545177, 1.387660016737271, 1.3142659582541292, 1.535903553101811, 1.0720883168748223, 1.420435520, 1.2415009026505173, 1.1940904449702439, 1.2293317304900846, 1.375106364184427, 1.3119949816195466, 1.55918392559895, 1.13468020639644, 1.38621293624]
nbr_y_zero = [], [], [], [], [], []]
```

```
train_sizes = [5, 10, 15, 20, 25, 30]
plt.plot(train_sizes, ave_mape_raw[:6], label="raw")
if base: plt.plot(train_sizes, ave_mape_basevsg[:6], label="base VSG")
if PSO: plt.plot(train_sizes, ave_mape_psovsg[:6], label="PSO VSG")
if DE: plt.plot(train_sizes, ave_mape_devsg[:6], label="DE VSG")
plt.xlabel("Training set size")
plt.ylabel("MAPE value")
plt.legend()
plt.title("MAPE comparison")
plt.savefig(f"{normalize}_mape_psodata_knn_mae.pdf")
plt.show()

plt.plot(train_sizes, ave_rmse_raw[:6], label="raw")
if base: plt.plot(train_sizes, ave_rmse_basevsg[:6], label="base VSG")
if PSO: plt.plot(train_sizes, ave_rmse_psovsg[:6], label="PSO VSG")
if DE: plt.plot(train_sizes, ave_rmse_devsg[:6], label="DE VSG")
plt.xlabel("Training set size")
plt.ylabel("RMSE value")
plt.legend()
plt.title("RMSE comparison")
plt.savefig(f"{normalize}_rmse_psodata_knn_mae.pdf")
plt.show()
```

```
ave_mape_raw = [48634709101357.63, 44865846357379.96, 33439347923351.5, 34072271986972.453, 36756781000760.625, 4548247988
ave_mape_psovsq = [48510468280764.11, 48140644009058.92, 36572329758325.89, 37571091582147.75, 39283326581919.586, 51050134
ave_mape_basevsq = [45437957272614.266, 41938266914722.47, 34665959570619.105, 34758145304651.348, 38879711498780.22, 54168
ave_mape_devsq = [49018024792296.71, 47255443020225.53, 37331385725032.88, 36216832577078.914, 39475251916899.34, 550640956
ave_rmse_raw = [0.2635816710967656, 0.22949773012572436, 0.22462693801603448, 0.21223014235034712, 0.2079637415763943, 0.21
ave_rmse_psovsq = [0.2589958820729695, 0.2272082580544406, 0.23284812086593393, 0.21761949795751176, 0.22324044593390502, 0.2
ave_rmse_basevsq = [0.2643863175711375, 0.22623578118431695, 0.221122788010789302, 0.2091489237714602, 0.2128015531516782, 0.2
ave_rmse_devsq = [0.2647619060012822, 0.23298710044801485, 0.21411112541587982, 0.20523404276227047, 0.2037920115859593, 0.
mape_raw = [[44514524667801.15, 78810869475305.52, 47141400568965.086, 42835987706480.51, 36884184965381.8, 829417569603097.
mape_psovsq = [[53673228530897.74, 71253779197053.16, 473636924107846.06, 43881227138482.12, 40729435301124.02, 799497946911
```

```

mape_basevsg = [[30405858933953.68, 82888267672592.9, 46498918158872.734, 33749844401493.03, 36127580411681.164, 8362810981
mape_devsg = [[36843312245940.625, 71664802611658.25, 47437842595923.375, 38481206109839.38, 41218989446340.945, 9163043719
rmse_raw = [[0.2694702424438907, 0.2801831260450049, 0.25284120629827206, 0.2868521586900544, 0.2808161865122947, 0.3158508
rmse_psovs = [[0.26409475390611814, 0.2562260909892597, 0.25076289788614636, 0.27353897763359425, 0.26325525835766284, 0.2
rmse_basevsg = [[0.28684218940049955, 0.28581377135710995, 0.23893634236439296, 0.2697947096762925, 0.268241601914736, 0.33
rmse_devsg = [[0.2760071917967346, 0.23975321864725763, 0.2485551937343965, 0.26596944045135784, 0.26345270328353204, 0.377

```

```

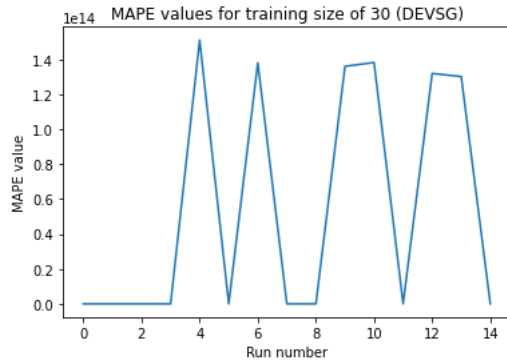
In [ ]: nbr_y_zero = [[0, 0, 0, 0, 13, 0, 13, 0, 0, 13, 13, 0, 13, 13, 0],
[0, 0, 0, 0, 13, 0, 13, 0, 0, 13, 13, 0, 13, 13, 0]]

```

```

In [ ]: plt.plot(mape_devsg[5], label="DEVSG")
plt.title(f"MAPE values for training size of {train_size} (DEVSG)")
plt.xlabel("Run number")
plt.ylabel("MAPE value")
plt.savefig("detailsmapede.pdf")
plt.show()

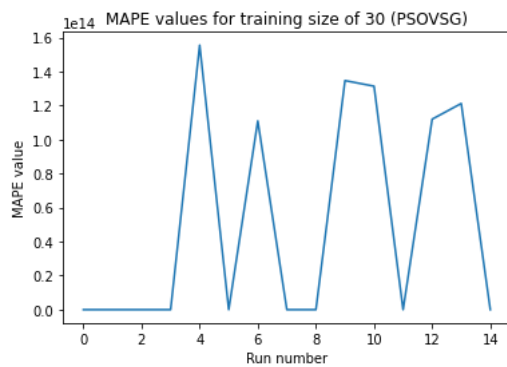
```



```

In [ ]: plt.plot(mape_psovs[5])
plt.title(f"MAPE values for training size of {train_size} (PSOVSG)")
plt.xlabel("Run number")
plt.ylabel("MAPE value")
plt.savefig("detailsmapeps.pdf")
plt.show()

```



```

In [ ]: plt.plot(nbr_y_zero[-1])
plt.title(f"Number of outputs close to 0 in the test set (training size of {train_size})")
plt.xlabel("Run number")
plt.ylabel("Number of outputs close to 0")
plt.savefig("detailsoutzero.pdf")
plt.show()

```

